DEVOPS UNIT 4

INFRASTRUCTURE AUTOMATION

Enterprise Student Record & Academic Management Platform

Terraform • Ansible • Docker • Flask • AWS

SUBMITTED BY
Name: shreya kardile
Roll No.: 2125UMLF1004
Class/Division: AIML
Faculty: Karan gaikwad

Infrastructure Automation for the Enterprise Student Record & Academic Management Platform

Unit: Unit 4 — Infrastructure Automation (AWS)

Tools: Terraform, Ansible, Docker, Flask, Nginx, Amazon EC2, Amazon VPC, Amazon S3

Student name: _____

Roll number: _____

Class/Division: _____

Faculty: _____

Submission date: _____

1. Aim

To automate cloud infrastructure provisioning and server configuration for an Enterprise Student Record and Academic Management Platform using Terraform and Ansible on AWS.

2. Problem statement

Manual cloud setup is slow, error-prone and difficult to reproduce. The project uses Infrastructure as Code to create repeatable AWS resources and Configuration Management to install software and deploy the application consistently.

3. Objectives

1. Provision a dedicated AWS VPC, public subnet, Internet Gateway and route table.

2. Configure a security group for restricted SSH and application/web access.

3. Launch an Ubuntu EC2 instance with an encrypted 20 GiB gp3 root volume.

4. Create a private S3 bucket with public-access blocking, server-side encryption, versioning and a lifecycle policy.

5. Use Ansible roles to configure common packages, Docker, Nginx and the Flask application.

6. Containerize the application and expose health, student and marks API endpoints.

7. Validate deployment using HTTP smoke tests.

4. Proposed architecture

A user sends HTTP requests to the EC2 public IP on port 80. Nginx receives the request and reverse-proxies it to the Flask/Gunicorn application running in a Docker container on port 8080. The app stores sample student and

marks data in SQLite on a persistent host-mounted directory. Terraform manages AWS resources; Ansible configures the server. A private S3 bucket is provisioned for future backup artifacts.

5. AWS infrastructure design

- **VPC:** `10.1.0.0/16`
- **Public subnet:** `10.1.1.0/24`
- **Internet Gateway:** attached to VPC
- **Route table:** `0.0.0.0/0` routed to the Internet Gateway
- **EC2:** Ubuntu 22.04 LTS, default `t2.micro`
- **Root volume:** 20 GiB gp3, encrypted
- **Security group:** SSH 22 from administrator CIDR; HTTP 80 and API 8080
- **S3 backup bucket:** private, AES-256 encryption, versioning and noncurrent-version lifecycle

SSH access is restricted to a configurable administrator CIDR rather than opened to all IP addresses. Port 8080 is public for direct API testing because the supplied assignment brief specifies that port.

6. Terraform implementation

Terraform uses HCL to declare the desired infrastructure. `terraform init` downloads provider plugins, `terraform fmt` formats configuration, `terraform validate` checks syntax and consistency, `terraform plan` previews changes and `terraform apply` provisions resources. Terraform outputs the EC2 public IP and S3 bucket name for subsequent configuration.

Terraform state contains infrastructure metadata and must be protected. This project uses local state for a straightforward first run. A shared team deployment should configure a remote S3 backend and appropriate state locking before concurrent use.

7. Ansible implementation

Ansible connects to the EC2 instance over SSH and executes reusable roles:

- **common:** refresh package cache, install utilities, and configure UFW rules.
- **docker:** install Docker Engine and enable the Docker service.
- **nginx:** install Nginx, deploy a Jinja2 reverse-proxy configuration and validate/reload Nginx.
- **library_app:** copy application files, build the Docker image, run the container with restart policy and persistent data mount, and check `/api/health`.

Ansible is agentless and uses SSH. Modules are designed for idempotent configuration. The deployment role rebuilds the application image and recreates the container when the playbook is run to keep deployment predictable.

8. Application endpoints

- `GET /api/health` — returns service health.
- `GET /api/students` — returns the sample student record.

- `POST /api/marks` — validates and stores marks for an existing student.
- `GET /api/marks` — lists stored marks.

Example request:

```
curl -X POST http://<PUBLIC_IP>/api/marks \
  -H "Content-Type: application/json" \
  -d '{"student_id":1,"subject":"DevOps","marks":88,"max_marks":100}'
```

Expected health response:

```
{"status":"UP","service":"student-record-api"}
```

9. Deployment and validation procedure

1. Configure AWS CLI credentials and create an EC2 key pair.

2. Set `key_name` and `admin_cidr` in `terraform/terraform.tfvars`.

3. Run `terraform init`, `terraform fmt -recursive`, `terraform validate`, `terraform plan` and review the plan.

4. Run `terraform apply` and record the public IP output.

5. Update `ansible/inventory.ini` with the public IP and private key path.

6. Install the Ansible collections listed in `ansible/requirements.yml`.

7. Run Ansible ping and the `ansible/site.yml` playbook.

8. Test `/api/health`, `/api/students`, and `POST /api/marks` with curl.

9. Save actual terminal output and screenshots as deployment evidence.

10. Smoke test commands

```
curl -f http://<PUBLIC_IP>/api/health
curl -f http://<PUBLIC_IP>/api/students
curl -f http://<PUBLIC_IP>/api/marks
curl -f -X POST http://<PUBLIC_IP>/api/marks \
  -H "Content-Type: application/json" \
  -d '{"student_id":1,"subject":"DevOps","marks":88,"max_marks":100}'
```

Expected outcomes: health returns HTTP 200 and status `UP`; students returns HTTP 200 with the seeded sample; a valid marks request returns HTTP 201; invalid payloads return HTTP 400; a non-existent student returns HTTP 404.

11. Security and cost considerations

- Keep AWS credentials and private keys out of Git.
- Restrict SSH to the administrator's public IP.
- Review the Terraform plan before applying.
- AWS resources may incur charges; destroy the stack after assessment if it is no longer needed.

- The S3 bucket is private and encrypted. The current starter creates the bucket but does not yet automate copying SQLite backups into it.
- The example uses HTTP, not HTTPS, and SQLite is suitable only for a small demonstration. Production deployment should add TLS, stronger authentication/authorization, monitored backups, secrets management and a managed database.

12. Limitations

The sample API demonstrates infrastructure automation and basic academic data endpoints. It is not a complete institutional information system and should not store real student personal data. Cloud provisioning and smoke-test results must be verified in the student's own AWS environment; no successful deployment is claimed by this report.

13. Conclusion

The project demonstrates the two-tier DevOps workflow: Terraform provisions cloud resources declaratively, and Ansible configures the operating system and deploys the containerized application. The combination improves repeatability, reviewability and operational consistency compared with manual setup.

14. References

1. Supplied CAREERTIQ Unit 4 Trainer Implementation Manual: *Infrastructure Automation using Terraform and Ansible on AWS*.

2. Terraform documentation: https://developer.hashicorp.com/terraform

3. Ansible documentation: https://docs.ansible.com/

4. AWS documentation: https://docs.aws.amazon.com/

5. Docker documentation: https://docs.docker.com/